ОТЧЁТ

по лабораторной работе №17

«Асинхронный Telegram-бот на aiogram 3»

Вариант 20. Интернет-провайдер

1. Цель работы

Научиться выносить операции личного кабинета в сервис и подключать их к обработчикам aiogram 3: текущий тариф, подключение услуги, история платежей и смена тарифа.

2. Постановка индивидуального задания

Вариант 20. Сущности: клиент, тариф, услуга, платёж. Функции бота: показать текущий тариф, подключить услугу, показать историю платежей и изменить тариф.

Сам бот запускается с переменной BOT_TOKEN. Тот же сервис прогнан локальным диалогом, чтобы показать сценарий без доступа к Telegram.

3. Краткие теоретические сведения

aiogram 3 принимает обновления Telegram и передаёт их в обработчик по фильтру: команда /start, текст кнопки или команда /service. Обработчик объявлен async, потому что отправка ответа — операция ожидания сети.

Клавиатура ReplyKeyboardMarkup показывает абоненту четыре действия. Предметные правила не живут в обработчике: их выполняет класс Cabinet. Так сценарий можно проверить и без бота.

4. Описание программы

Команда /start регистрирует абонента на тарифе «Домашний 100» и записывает аванс. «Текущий тариф» печатает плату, услуги и состояние. /service ТВ добавляет услугу и увеличивает плату на 250 рублей. /tariff меняет базовый тариф и сохраняет уже подключённые услуги. Заблокированному из-за задолженности абоненту смена тарифа отказана.

Локальный диалог повторяет эти шаги от лица Иванова: старт, текущий тариф, услуга ТВ, история и переход на «Домашний 300».

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — bot.py

```
"""
Лабораторная работа №17. Вариант 20.
Telegram-бот личного кабинета провайдера (aiogram 3).
Токен берётся из переменной BOT_TOKEN.
"""

import asyncio
import os

from aiogram import Bot, Dispatcher, F
from aiogram.filters import Command, CommandStart
from aiogram.types import KeyboardButton, Message, ReplyKeyboardMarkup

from services import SERVICE_PRICE, TARIFFS, cabinet

keyboard = ReplyKeyboardMarkup(
    keyboard=[
        [KeyboardButton(text="Текущий тариф"), KeyboardButton(text="История платежей")],
        [KeyboardButton(text="Подключить услугу"), KeyboardButton(text="Сменить тариф")],
    ],
    resize_keyboard=True,
)



def build_dispatcher() -> Dispatcher:
    dp = Dispatcher()

    @dp.message(CommandStart())
    async def start(message: Message):
        name = message.from_user.full_name if message.from_user else "Абонент"
        text = cabinet.register(message.from_user.id, name)
        await message.answer(text + "\nВыберите действие.", reply_markup=keyboard)

    @dp.message(F.text == "Текущий тариф")
    async def current(message: Message):
        await message.answer(cabinet.current_tariff(message.from_user.id))

    @dp.message(F.text == "История платежей")
    async def history(message: Message):
        await message.answer(cabinet.history(message.from_user.id))

    @dp.message(Command("service"))
    async def service_cmd(message: Message):
        parts = (message.text or "").split(maxsplit=1)
        if len(parts) < 2:
            await message.answer("Формат: /service антивирус\nДоступны: " + ", ".join(SERVICE_PRICE))
            return
        await message.answer(cabinet.connect_service(message.from_user.id, parts[1].strip()))

    @dp.message(F.text == "Подключить услугу")
    async def service_hint(message: Message):
        await message.answer("Напишите /service и название. Например: /service ТВ")

    @dp.message(Command("tariff"))
    async def tariff_cmd(message: Message):
        parts = (message.text or "").split(maxsplit=1)
        if len(parts) < 2:
            await message.answer("Формат: /tariff Домашний 300\nДоступны: " + ", ".join(TARIFFS))
            return
        await message.answer(cabinet.change_tariff(message.from_user.id, parts[1].strip()))

    @dp.message(F.text == "Сменить тариф")
    async def tariff_hint(message: Message):
        await message.answer("Напишите /tariff и название. Например: /tariff Домашний 300")

    return dp


async def main():
```

```
    token = os.environ.get("BOT_TOKEN", "")
    if not token:
        print("Задайте BOT_TOKEN. Локальная демонстрация без Telegram: python demo.py")
        return
    bot = Bot(token)
    await build_dispatcher().start_polling(bot)



if __name__ == "__main__":
    asyncio.run(main())
```

Листинг — demo.py

```
"""
Локальный диалог с той же логикой, что и у бота.
Нужен, чтобы показать сценарий без токена Telegram.
"""

import sys

from services import Cabinet

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")



def say(who, text):
    print(f"{who}: {text}")



def main():
    cabinet = Cabinet()
    user_id = 20
    script = [
        ("/start", lambda: cabinet.register(user_id, "Иванов")),
        ("Текущий тариф", lambda: cabinet.current_tariff(user_id)),
        ("/service ТВ", lambda: cabinet.connect_service(user_id, "ТВ")),
        ("История платежей", lambda: cabinet.history(user_id)),
        ("/tariff Домашний 300", lambda: cabinet.change_tariff(user_id, "Домашний 300")),
        ("Текущий тариф", lambda: cabinet.current_tariff(user_id)),
    ]
    say("Бот", "Личный кабинет провайдера. Команды совпадают с aiogram-обработчиками.")
    for command, action in script:
        say("Абонент", command)
        say("Бот", action())
    print("Диалог завершён.")



if __name__ == "__main__":
    main()
```

Листинг — services.py

```
"""Бизнес-логика личного кабинета провайдера."""

from dataclasses import dataclass, field



@dataclass
class Account:
    name: str
    tariff: str
    fee: float
    services: list[str] = field(default_factory=list)
    payments: list[tuple[str, float]] = field(default_factory=list)
    blocked: bool = False



TARIFFS = {
```

```
    "Домашний 100": 500,
    "Домашний 300": 800,
    "Корпоративный 500": 2500,
}
SERVICE_PRICE = {"антивирус": 100, "ТВ": 250, "белый IP": 200}



class Cabinet:
    def __init__(self):
        self.accounts: dict[int, Account] = {}

    def register(self, user_id: int, name: str) -> str:
        self.accounts[user_id] = Account(name, "Домашний 100", TARIFFS["Домашний 100"],
payments=[("аванс", 500)])
        return f"{name}, подключён тариф «Домашний 100»."

    def current_tariff(self, user_id: int) -> str:
        account = self._get(user_id)
        services = ", ".join(account.services) if account.services else "нет"
        state = "заблокирован" if account.blocked else "активен"
        return (
            f"Тариф «{account.tariff}», {account.fee:.0f} руб./мес, "
            f"услуги: {services}, состояние: {state}."
        )

    def connect_service(self, user_id: int, service: str) -> str:
        account = self._get(user_id)
        if service not in SERVICE_PRICE:
            return "Такой услуги нет. Доступны: " + ", ".join(SERVICE_PRICE)
        if service in account.services:
            return f"Услуга «{service}» уже подключена."
        account.services.append(service)
        account.fee += SERVICE_PRICE[service]
        return f"Услуга «{service}» подключена. Новая плата {account.fee:.0f} руб."

    def history(self, user_id: int) -> str:
        account = self._get(user_id)
        if not account.payments:
            return "Платежей пока нет."
        lines = [f"{title}: {amount:.0f} руб." for title, amount in account.payments]
        return "История платежей:\n" + "\n".join(lines)

    def change_tariff(self, user_id: int, tariff: str) -> str:
        account = self._get(user_id)
        if account.blocked:
            return "Смена тарифа недоступна: есть задолженность, абонент заблокирован."
        if tariff not in TARIFFS:
            return "Тариф не найден. Доступны: " + ", ".join(TARIFFS)
        extra = sum(SERVICE_PRICE[name] for name in account.services)
        account.tariff = tariff
        account.fee = TARIFFS[tariff] + extra
        account.payments.append((f"смена на {tariff}", 0))
        return f"Тариф изменён на «{tariff}». К оплате {account.fee:.0f} руб."

    def pay(self, user_id: int, amount: float) -> str:
        account = self._get(user_id)
        account.payments.append(("оплата", amount))
        if amount >= account.fee:
            account.blocked = False
        return f"Платёж {amount:.0f} руб. принят."

    def _get(self, user_id: int) -> Account:
        if user_id not in self.accounts:
            raise KeyError("Сначала отправьте /start")
        return self.accounts[user_id]


cabinet = Cabinet()
```

6. Примеры работы программы

Снимок показывает локальный диалог, который вызывает те же методы, что и обработчики бота.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python17> python demo.py

Бот: Личный кабинет провайдера. Команды совпадают с aiogram-обработчиками.
Абонент: /start
Бот: Иванов, подключён тариф «Домашний 100».
Абонент: Текущий тариф
Бот: Тариф «Домашний 100», 500 руб./мес, услуги: нет, состояние: активен.
Абонент: /service ТВ
Бот: Услуга «ТВ» подключена. Новая плата 750 руб.
Абонент: История платежей
Бот: История платежей:
аванс: 500 руб.
Абонент: /tariff Домашний 300
Бот: Тариф изменён на «Домашний 300». К оплате 1050 руб.
Абонент: Текущий тариф
Бот: Тариф «Домашний 300», 1050 руб./мес, услуги: ТВ, состояние: активен.
Диалог завершён.
```

Рисунок 1 – Диалог: тариф, услуга ТВ, история платежей и смена тарифа

7. Ответы на контрольные вопросы

1. Почему обработчики асинхронные?

Ожидание сети Telegram не должно блокировать весь процесс. async даёт боту обслуживать другие обновления в это время.

2. Зачем выносить логику из bot.py?

Правила тарифа проверяются без Telegram. Обработчик только достаёт текст сообщения и отправляет ответ сервиса.

3. Как абонент выбирает действие?

После /start бот показывает клавиатуру: текущий тариф, история, услуга и смена тарифа.

4. Что происходит при смене тарифа?

Базовая цена заменяется, плата за уже подключённые услуги сохраняется, в историю пишется отметка о смене.

5. Почему для снимка использован demo.py?

Без токена бота Telegram не отвечает. Демонстрация вызывает тот же Cabinet и показывает сценарий варианта.

8. Вывод

В лабораторной работе №17 личный кабинет провайдера варианта 20 реализован сервисом и подключён к обработчикам aiogram 3. Сценарий текущего тарифа, услуги, истории платежей и смены тарифа проверен локальным диалогом.